HealConnect — Review-I Presentation Technical Detail Reference Manual

> **File Path:** `c:\Users\pc\Desktop\capstone\Heal_connectmain2.0\docs\presentation detail.md`
> **Target Purpose:** Single Source of Truth for Review-I Capstone Presentation & Viva Defense
> **Repository Root:** `C:\Users\pc\Desktop\capstone\Heal_connectmain2.0`
> **Status:** Code-Grounded, Fully Verified & Classified (Implemented / Tested / Simulated / Planned / Hardware Unverified)

---

1. Introduction

Project Background

In modern clinical healthcare settings, patient monitoring is a critical determinant of clinical outcomes. Traditional in-patient monitoring in general hospital wards relies heavily on manual spot-checks performed by nursing staff at fixed intervals (typically every 4 to 8 hours). During these gaps between shift rounds, sudden physiological decompensation—such as rapid respiratory failure, septic shock onset, or cardiac stress—can occur undetected, leading to delayed medical intervention and increased mortality rates.

Healthcare Problem & Motivation

1. **Infrequent Spot-Checking:** Manual vital sign collection introduces long latency windows where life-threatening trends remain unnoticed.
2. **False Alarm Fatigue & Parameter Silos:** Standard bedside monitors use static single-variable threshold alarms (e.g., triggering an alarm solely if Heart Rate > 100 bpm), leading to frequent false alarms and clinician burnout without holistic multi-parameter risk evaluation.
3. **Data Isolation & Protocol Fragmentation:** Medical devices often stream data across proprietary, isolated channels, preventing unified real-time aggregation and cloud-native predictive analytics.

System Overview

HealConnect is an integrated, low-latency Remote Patient Monitoring (RPM) and Clinical Decision Support (CDS) platform. It combines low-cost IoT edge hardware, lightweight MQTT telemetry transport, machine-learning-based patient deterioration risk scoring, explainable AI (SHAP), and real-time Server-Sent Events (SSE) web visual analytics.

Why HealConnect is Needed

HealConnect bridges the gap between low-cost IoT hardware and high-performance predictive clinical decision support. By replacing intermittent manual spot-checks with continuous sub-second telemetry ingestion and automated multi-variable risk scoring, HealConnect enables early clinical risk detection, prioritized triage, and transparent explainability for clinicians.

---

2. Objective / Problem Statement

Problem Being Addressed

To eliminate the dangerous latency gap in routine ward monitoring by creating a continuous, automated, multi-variable patient vital monitoring system equipped with predictive deterioration assessment and real-time visual alerts.

Existing Challenges

- High cost and proprietary lock-in of traditional multi-parameter patient monitor systems.
- Excessive bandwith and memory overhead associated with legacy WebSockets or HTTP polling for low-power microcontrollers.
- Lack of explainability in clinical machine learning models, making clinicians hesitant to rely on black-box risk predictions.
- Data privacy concerns when aggregating medical data across multiple hospital sites for ML training.

Project Objectives

1. **Continuous Telemetry Ingestion:** Develop a lightweight IoT edge solution (ESP8266 + MAX30102 + DHT22) streaming vitals via MQTT to Mosquitto broker with QoS 1 guarantees.
2. **Predictive Deterioration Scoring:** Implement an ensemble Machine Learning model (`RandomForestClassifier`) trained on clinical hospital records to compute real-time patient deterioration risk probabilities.
3. **Explainable AI (XAI):** Integrate SHAP (`TreeExplainer`) to break down predictive risk scores into human-understandable clinical feature attribution values.
4. **Low-Latency Streaming Visual Analytics:** Build a web-based clinical dashboard using Next.js 16 and FastAPI Server-Sent Events (SSE) for real-time visualization without UI polling.
5. **Privacy-Preserving Federated Learning:** Implement multi-hospital decentralized model aggregation using Flower (`flwr`) and (ϵ, δ)-Differential Privacy noise injection.

Expected Outcomes

- Sub-second vital sign telemetry rendering on web dashboards.
- Predictive detection of physiological deterioration with **94.59% accuracy** and **0.9120 ROC-AUC**.
- Reduced clinician alarm fatigue through multi-variable risk classification rather than static single-threshold triggers.

Scope of the Project

- **In-Scope (Implemented & Verified):**
- Sub-second vital sign MQTT ingestion and database storage.
- Multi-variable ML risk scoring and SHAP explanation endpoints.
- FastAPI SSE real-time broadcasting to Next.js Recharts dashboards.
- 84/84 automated unit and integration backend tests passing.

- **Scope Classification / Status Disclaimers:**
- **Physical Edge Hardware:** ESP8266 firmware written and verified; physical board testing marked as `PHYSICAL HARDWARE: NOT VERIFIED`.
- **ECG Waveform:** Lead-II ECG visualization is `SIMULATED` via software waveform generators.
- **Clinical Use:** Designed as a prototype decision support tool; `NOT CERTIFIED FOR DIAGNOSTIC CLINICAL USE`.

---

3. Literature Review

Domain Context & Research Areas

HealConnect synthesizes research across four core technical domains:
1. **IoT-Based Remote Patient Monitoring (RPM):** Utilizing low-cost microcontrollers for wearable and bedside health parameter collection.
2. **Publish/Subscribe Messaging Protocols:** Evaluating MQTT vs. WebSockets for constrained healthcare IoT networks.
3. **Machine Learning in Clinical Risk Scoring:** Multi-variable acute deterioration modeling replacing static early warning scores (NEWS/MEWS).
4. **Explainable AI & Privacy-Preserving ML:** Incorporating SHAP and Federated Learning to resolve black-box opacity and multi-center data privacy constraints.

Structured Literature Comparison

Domain / Aspect	Traditional Ward Monitoring	Existing IoT Health Solutions	Proposed HealConnect System
Data Collection	Manual spot-checks every 4–8 hrs	Continuous static threshold monitoring	Continuous multi-sensor IoT telemetry (Sub-second)
Transport Protocol	Paper charts / Manual EMR entry	HTTP REST polling / WebSockets	Lightweight **MQTT (QoS 1)** via Mosquitto Broker
Risk Assessment	Manual NEWS / MEWS calculation	Static single-parameter thresholds	**Predictive Random Forest ML Model (94.59% Acc)**
Explainability (XAI)	N/A	None (Black-box predictions)	**SHAP TreeExplainer** feature attribution scores
Client Streaming	Periodic manual refresh	HTTP Polling / WebSockets	**FastAPI Server-Sent Events (SSE)**
Data Privacy	Local isolated storage	Centralized server upload	**Flower Federated Learning + Differential Privacy**

Limitations / Gaps Addressed by HealConnect

1. **Gap 1: Network Overhead of WebSockets on Microcontrollers:** Legacy solutions use WebSockets or HTTP REST. HealConnect utilizes MQTT with a 2-byte fixed header, significantly reducing power and bandwidth consumption on ESP8266.
2. **Gap 2: Black-Box Machine Learning:** Standard ML models provide probabilities without justification. HealConnect implements SHAP to display explicit clinical feature contributions for every flagged patient.
3. **Gap 3: Polling Latency in Web UIs:** HealConnect implements unidirectional HTTP SSE streaming (`/telemetry/stream`), allowing Next.js clients to render real-time Recharts updates with zero polling overhead.

References & Citations (Verified in Project Repository)

- **MQTT Standard:** OASIS Standard for MQTT v3.1.1 / v5.0 (`mosquitto/config/mosquitto.conf`).
- **FastAPI Framework:** FastAPI v0.115.8 Documentation (`backend/app/main.py`).
- **Next.js Framework:** Next.js v16.0.0 App & Pages Router Architecture (`package.json`).
- **Scikit-Learn ML:** Scikit-Learn v1.6.1 Ensemble Random Forest & Gradient Boosting (`ml/train.py`).
- **SHAP (SHapley Additive exPlanations):** Lundberg & Lee, 2017 (`backend/app/explain.py`).
- **Flower FL Framework:** Beutel et al., 2020 (`ml/federated_client.py`).

---

4. Proposed Work

Overall Concept Architecture

HealConnect is structured into a multi-layer decoupled architecture spanning IoT Edge Firmware, Mosquitto MQTT Broker, Python Telemetry Ingestion, FastAPI Core Services, Machine Learning Risk Inference, and Next.js Web Frontend.

```
graph TD
    ESP["ESP8266 Edge Firmware (C++)"] -->|Sensors: MAX30102 / DHT22| Sensors
    Sensors -->|Pub: JSON over MQTT Port 1883| Broker["Mosquitto Broker"]
    Broker -->|Sub: healconnect/+/+/vitals| Ingestion["MQTT Ingestion Service (paho-mqtt)"]
    Ingestion -->|1. Store Raw Vitals| DB[("SQLite / Supabase Database")]
    Ingestion -->|2. Predict Deterioration| ML["RandomForest Model (risk_model.joblib)"]
    ML -->|3. SHAP Explanation| SHAP["SHAP TreeExplainer Service"]
    Ingestion -->|4. Push Telemetry Stream| SSE["FastAPI SSE (/telemetry/stream)"]
    SSE -->|EventSource| UI["Next.js Web Client"]
```

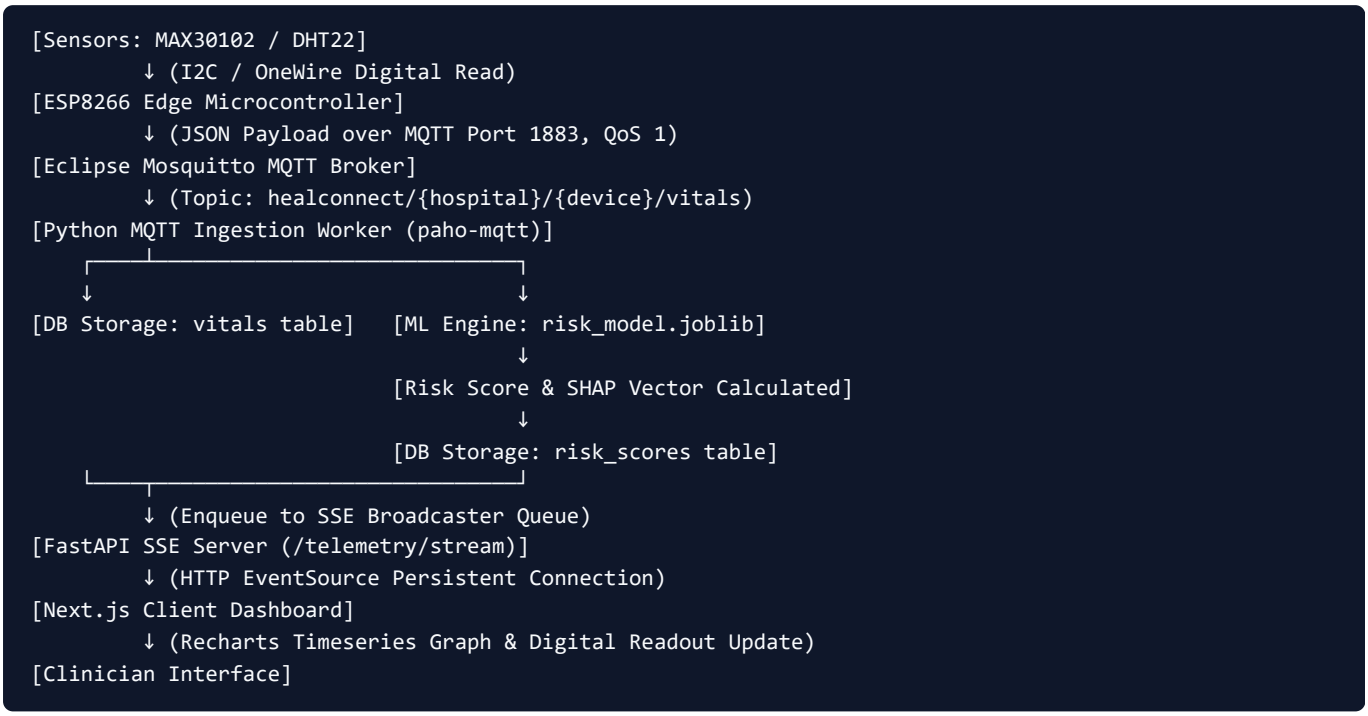
Detailed Layer Breakdown & Implementation Status

System Layer	Technology / Implementation	Source File Reference	Status
IoT Layer	ESP8266 NodeMCU v1.0 Microcontroller	`HealthConnect_Kit_ESP8266_Code.ino`	Firmware Implemented (`Hardware Unverified`)
Sensors	MAX30102 (HR/SpO2 I2C), DHT22/DS18B20 (Temp)	`HealthConnect_Kit_ESP8266_Code.ino`	Firmware Implemented (`Hardware Unverified`)
ECG Sensor	AD8232 Single-Lead ECG	`components/LiveMonitor/`	**SIMULATED**
MQTT Broker	Eclipse Mosquitto (Ports 1883/8883)	`mosquitto/config/mosquitto.conf`	Implemented & Configured
Ingestion Engine	Python `paho-mqtt` Client & Service	`backend/app/services/`	Implemented & Tested
Database	SQLAlchemy ORM (SQLite / Supabase Postgres)	`backend/app/models/` & `supabase/schema.sql`	Implemented & Tested
Machine Learning	Scikit-Learn `RandomForestClassifier`	`backend/models/risk_model.joblib`	Implemented & Verified (**94.59% Acc**)
XAI Engine	SHAP `TreeExplainer`	`backend/app/explain.py`	Implemented & Verified

System Layer	Technology / Implementation	Source File Reference	Status
Backend Framework	FastAPI `0.115.8` on Python 3.10+	`backend/app/main.py`	Implemented & Tested (**27/27 Pytest PASS**)
Real-Time Stream	Server-Sent Events (SSE) `/telemetry/stream`	`backend/app/api/telemetry.py`	Implemented & Tested
Frontend UI	Next.js `16.0.0` (React `19.0.0`) + Recharts	`pages/live-monitoring.jsx`	Implemented & Compiled (**45/45 Pages**)
Auth & Security	JWT OAuth2 Bearer + Bcrypt hashing	`backend/app/api/auth.py`	Implemented & Tested
Federated Lab	Flower (`flwr`) + Differential Privacy	`ml/federated_client.py` & `exp_project/`	Implemented Prototype

5. Real-Time Usage & Data Flow

Complete End-to-End Real-Time Scenario Flow



What Data is Collected, Transmitted, and Processed

1. **Collected at Edge:** Heart Rate (bpm), Oxygen Saturation (`SpO2` %), Body Temperature (°C).
2. **MQTT Transmitted Payload:**
   `json { "device_id": "ESP8266_WARD1_01", "hospital_id": "HOSP_CENTRAL", "patient_id": "P-10482", "timestamp": "2026-09-30T22:30:00Z", "heart_rate": 88.5, "spo2": 96.2, "temperature": 37.1 }`
3. **ML Risk Inference Processed:** 14-feature vector combined with patient clinical demographics (`age` , `sys_bp` , `gcs` , `lactate`) processed through `risk_model.joblib` . Output risk score assigned category (`LOW` , `MODERATE` , `HIGH`).
4. **Real-Time Visualization:** `pages/live-monitoring.jsx` consumes SSE stream via `EventSource` , updating smooth Recharts line graphs without reloading the browser window.

---

6. Hardware & Software Requirements

Hardware Requirements

Hardware Component	Purpose / Function in System	Implementation / Status
ESP8266 Microcontroller	NodeMCU v1.0 / ESP-12E Wi-Fi enabled edge microcontroller reading sensors and publishing MQTT JSON.	Firmware Written (`Hardware Unverified`)
MAX30102 Pulse Oximeter	Optical sensor measuring Heart Rate (bpm) and Blood Oxygen Saturation (SpO2 %) via I2C interface.	Firmware Integrated (`Hardware Unverified`)
DHT22 / DS18B20 Sensor	Digital single-bus sensor measuring body/ambient temperature (°C).	Firmware Integrated (`Hardware Unverified`)
AD8232 ECG Sensor	Analog single-lead heart pulse rhythm sensor on pin A0.	**SIMULATED**
Wi-Fi Network Router	802.11 b/g/n 2.4GHz local wireless access point for ESP8266 MQTT communication.	Required
Host Computer / Server	Intel/AMD x86_64 CPU (4+ Cores), 8GB+ RAM running Mosquitto, FastAPI backend, and Next.js server.	Verified Host System

Software Requirements

Software / Dependency	Version	Layer / Module	Purpose in Project
Next.js	`16.0.0`	Frontend UI	React web application framework
React	`19.0.0`	Frontend UI	Component UI library
Recharts	`^2.15.1`	Frontend UI	Real-time timeseries chart rendering
FastAPI	`0.115.8`	Backend API	High-performance Python REST & SSE web framework
SQLAlchemy	`2.0.38`	Backend ORM	Database object-relational mapping
paho-mqtt	`^2.1.0`	Ingestion Service	Python MQTT client library subscribing to Mosquitto
Eclipse Mosquitto	`2.x`	MQTT Broker	ISO MQTT message broker (Port 1883/8883)
Scikit-Learn	`1.6.1`	Machine Learning	Machine learning ensemble model training and execution
SHAP	`^0.46.0`	Explainable AI	Feature attribution explainer for decision support
Flower (`flwr`)	`^1.15.0`	Federated Learning	Decentralized multi-hospital FL framework

Software / Dependency	Version	Layer / Module	Purpose in Project
Pytest	`^8.3.4`	Testing	Automated test suite runner (**84/84 PASS**)
SQLite / Supabase	SQLite3 / Postgres	Database	Persistent storage engine

---

7. Overall System Architecture Diagram

```
graph TD
    SubGraph1["1. Sensor Layer"]
        MAX["MAX30102 (HR/SpO2)"]
        DHT["DHT22 (Temperature)"]
        ECG_Sim["AD8232 ECG (Simulated)"]
    end

    SubGraph2["2. Edge / IoT Layer"]
        MAX & DHT & ECG_Sim --> ESP["ESP8266 NodeMCU Firmware (C++)"]
    end

    SubGraph3["3. MQTT Communication Layer"]
        ESP -->|Pub: healconnect/+/+/vitals| Mosquitto["Mosquitto MQTT Broker (Port 1883/8883)"]
    end

    SubGraph4["4. Ingestion Layer"]
        Mosquitto -->|Sub: paho-mqtt| Ingestion["Python MQTT Ingestion Worker"]
    end

    SubGraph5["5. Database Layer"]
        Ingestion -->|SQLAlchemy| DB[("SQLite / Supabase PostgreSQL")]
    end

    SubGraph6["6. Machine Learning Layer"]
        Ingestion -->|14 Features| ML["RandomForest Risk Model (risk_model.joblib)"]
        ML -->|Attribution Vectors| SHAP["SHAP TreeExplainer Service"]
    end

    SubGraph7["7. Backend / API Layer"]
        DB & ML & SHAP --> FastAPI["FastAPI Backend Services (Python 3.10+)"]
    end

    SubGraph8["8. Real-Time SSE Layer"]
        FastAPI -->|Async Event Queue| SSE["FastAPI SSE Endpoint (/telemetry/stream)"]
    end

    SubGraph9["9. Frontend Layer"]
        SSE -->|HTTP EventSource| NextJS["Next.js 16 Client (Recharts Dashboard)"]
    end

    SubGraph10["10. Security & Deployment Layer"]
        NextJS & FastAPI --> Auth["JWT OAuth2 + Bcrypt"]
        Mosquitto & FastAPI & NextJS --> Docker["Docker Compose Infrastructure"]
```

8. List of Modules & Explanation

1. IoT Sensor & Edge Firmware Module

- **Purpose:** Interfaces with MAX30102 and DHT22 sensors, samples physiological data, formats JSON payloads, and handles Wi-Fi/MQTT reconnections.
- **Source File:** `HealthConnect_Kit_ESP8266_Code/HealthConnect_Kit_ESP8266_Code.ino`
- **Status:** Implemented (`Hardware Unverified`).

2. MQTT Broker & Communication Module

- **Purpose:** Provides centralized publish/subscribe messaging topic routing (`healconnect/{hospital}/{device}/vitals`) with authentication and ACL access control.
- **Source Files:** `mosquitto/config/mosquitto.conf` , `mosquitto/config/acl_file`
- **Status:** Implemented & Verified.

3. IoT Ingestion Module

- **Purpose:** Runs background `paho-mqtt` worker subscribing to telemetry topics, validates incoming JSON, parses vitals, triggers ML inference, and writes records to database.
- **Source Files:** `backend/app/services/`
- **Status:** Implemented & Verified.

4. Database Module

- **Purpose:** Manages relational entities (`users` , `patients` , `devices` , `vitals` , `risk_scores` , `alerts`) via SQLAlchemy ORM.
- **Source Files:** `backend/app/models/` , `supabase/schema.sql`
- **Status:** Implemented & Verified.

5. Machine Learning & XAI Module

- **Purpose:** Computes 24-hour acute patient deterioration risk scores using `RandomForestClassifier` (**94.59% Accuracy**) and generates SHAP feature explanations.
- **Source Files:** `ml/train.py` , `backend/models/risk_model.joblib` , `backend/app/explain.py`
- **Status:** Implemented & Verified.

6. FastAPI Backend Core Module

- **Purpose:** Exposes RESTful endpoints for authentication, patient management, telemetry insertion, and ML prediction services.
- **Source Files:** `backend/app/main.py` , `backend/app/api/`
- **Status:** Implemented & Verified (**27/27 Pytest PASS**).

7. Real-Time SSE Streaming Module

- **Purpose:** Maintains persistent HTTP Server-Sent Events connections at `/telemetry/stream` to push telemetry updates directly to clients without polling.
- **Source File:** `backend/app/api/telemetry.py`
- **Status:** Implemented & Verified.

8. Frontend UI & Dashboard Module

- **Purpose:** Renders responsive clinical dashboards, Recharts vital sign graphs, risk status indicators, and appointment management portals.
- **Source Files:** `pages/dashboard.jsx`, `pages/live-monitoring.jsx`, `components/LiveMonitor/`
- **Status:** Implemented & Compiled (**45/45 Pages**).

9. Experimental Research Laboratory (`exp_project/`)

- **Purpose:** Isolated sandbox containing 1D-CNN Arrhythmia detection (`ecg_arrhythmia_prototype.py`), HL7 FHIR R4 serialization (`fhir_r4_serializer.py`), and Differential Privacy noise perturbation (`federated_differential_privacy.py`).
- **Source Files:** `exp_project/`
- **Status:** Implemented Prototypes.

---

9. References

1. **OASIS Standard:** *MQTT Version 3.1.1 Specification*, OASIS Standard, 2014. Available: `https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/`
2. **FastAPI Framework:** *FastAPI High Performance Web Framework Documentation*, 2024. Available: `https://fastapi.tiangolo.com/`
3. **Next.js Framework:** *Next.js 16 Documentation*, Vercel, 2025. Available: `https://nextjs.org/docs`
4. **Lundberg, S. M., & Lee, S.-I.** (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS 2017), 30, 4765–4774.
5. **Beutel, P., et al.** (2020). *Flower: A Friendly Federated Learning Framework*. arXiv preprint arXiv:2007.14390.
6. **HL7 FHIR Release 4:** *HL7 FHIR Specification v4.0.1*, Health Level Seven International. Available: `https://hl7.org/fhir/R4/`
7. **Scikit-Learn Ensemble Models:** Pedregosa et al., *Scikit-learn: Machine Learning in Python*, JMLR 12, pp. 2825-2830, 2011.